ANAND INSTITUTE OF HIGHER TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Mini Project Report

on

ELECTRICAL LINE FATIGUE

Submitted by

GOKUL R.V | Reg. No: 310123104020
FARSHATH AHAMED S | Reg. No: 310123104016
ADITHYA V.T | Reg. No: 310123104001
DEEPAK M | Reg. No: 310123104006

Academic Year: 2025 – 2026

DESCRIPTION

The Electrical Line Fatigue Monitoring System is an IoT-based solution designed to continuously monitor power line conditions in real time. The system uses an Arduino Uno microcontroller that reads temperature and current values from sensors and classifies the electrical line condition as SAFE, FATIGUE, or DANGER.

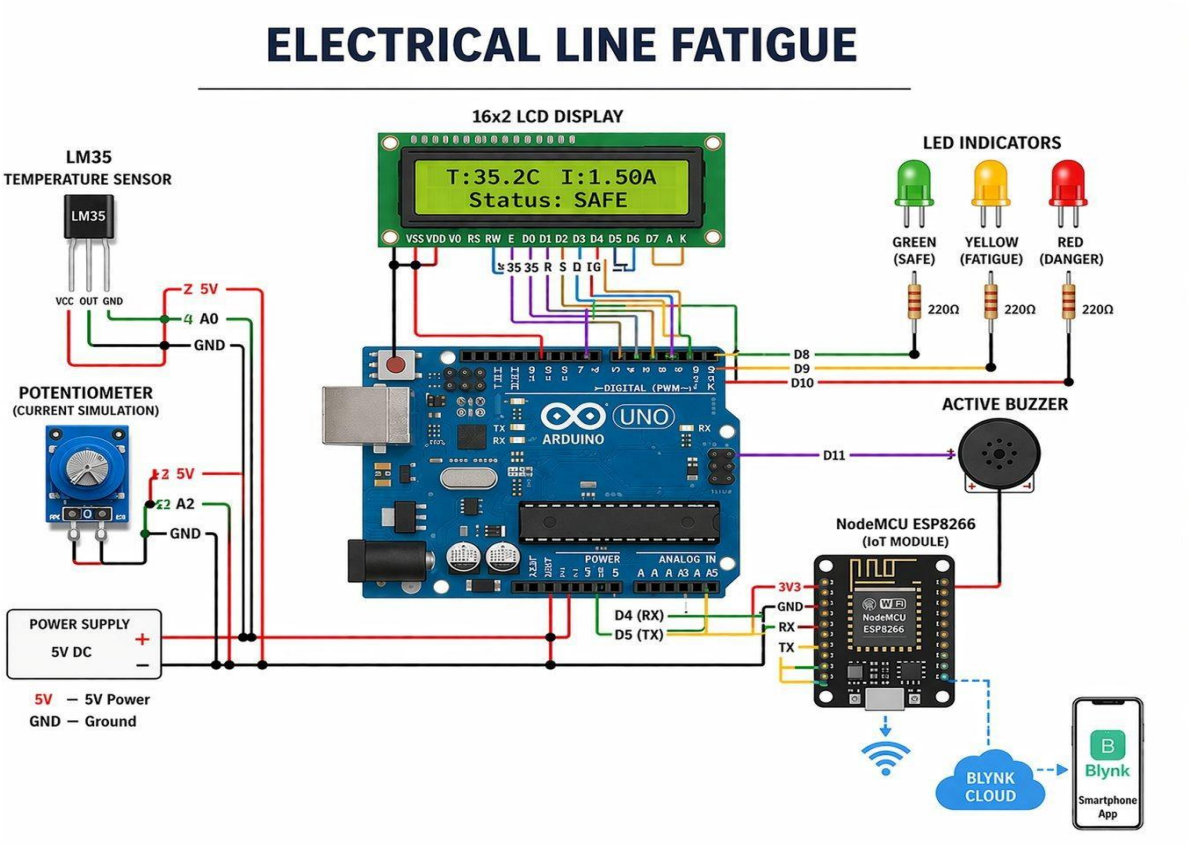
The primary objective is to detect overloading and overheating conditions early and notify users both locally (LCD display, LEDs, buzzer) and remotely via the Blynk smartphone application. A NodeMCU ESP8266 module handles WiFi communication and transmits live sensor data to the Blynk IoT cloud, enabling remote monitoring without requiring physical presence.

A potentiometer is used to simulate current variations for prototype demonstration. The system is particularly useful for electrical substations, industrial power lines, and residential distribution panels where overloading can cause equipment damage or fire hazards.

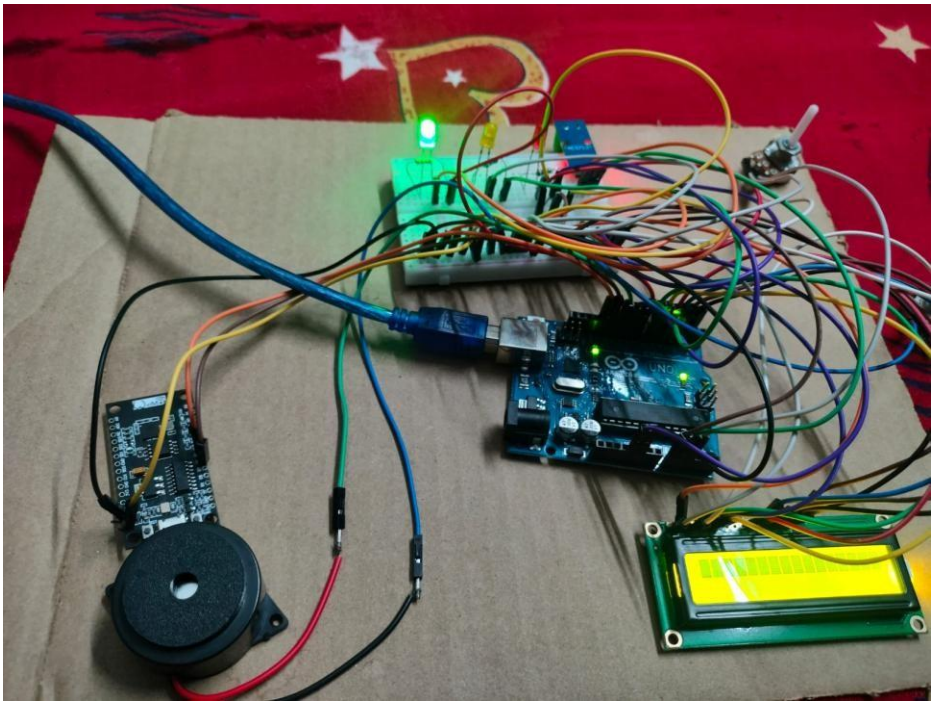
COMPONENTS USED

- Arduino Uno (ATmega328P microcontroller — main processing unit)
- NodeMCU ESP8266 (Wi-Fi enabled IoT module)
- LM35 Temperature Sensor (10 mV/°C, connected to A0)
- Potentiometer (Current simulation, connected to A2)
- 16×2 LCD Display (real-time local data display)
- LED Indicators — Green (SAFE), Yellow (FATIGUE), Red (DANGER)
- Active Buzzer (audio alert for FATIGUE and DANGER conditions)
- Breadboard and Jumper Wires
- 5V DC Power Supply
- Blynk IoT Platform (smartphone app — software component)

CIRCUIT DIAGRAM



HARDWARE SETUP



CODE

Arduino Code — Electrical Line Fatigue Monitor

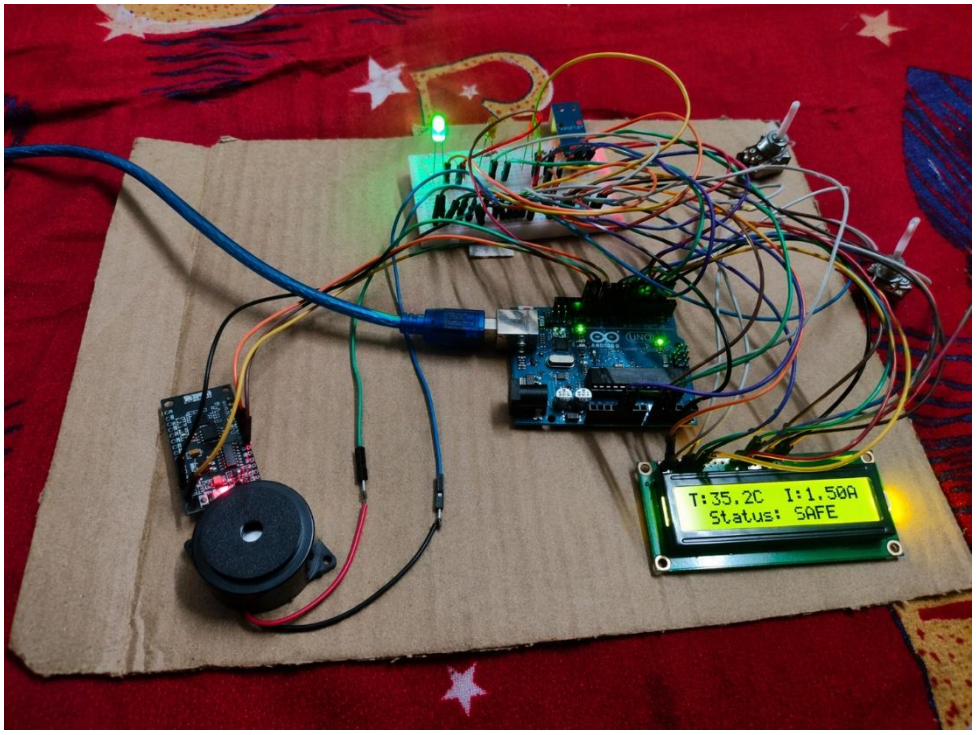
```
#include <LiquidCrystal.h>

// LCD
LiquidCrystal lcd(7, 6, 5, 4, 3, 2);

// Pins
const int TEMP_PIN    = A0;
const int CURRENT_PIN = A2;

const int LED_GREEN  = 8;
const int LED_YELLOW = 9;
const int LED_RED    = 10;
const int BUZZER     = 11;

// Thresholds
const float TEMP_FATIGUE  = 40.0;
const float TEMP_DANGER   = 60.0;
const float CURR_FATIGUE  = 2.0;
const float CURR_DANGER   = 4.0;

void setup()
{
  pinMode(LED_GREEN,  OUTPUT);
  pinMode(LED_YELLOW, OUTPUT);
  pinMode(LED_RED,    OUTPUT);
  pinMode(BUZZER,     OUTPUT);

  // Buzzer test
  digitalWrite(BUZZER, HIGH);
  delay(300);
  digitalWrite(BUZZER, LOW);

  lcd.begin(16, 2);
  lcd.setCursor(0, 0);
  lcd.print(" Line Fatigue ");
  lcd.setCursor(0, 1);
  lcd.print(" Monitor v1.0 ");
  delay(2000);
  lcd.clear();

  Serial.begin(9600);
}
```

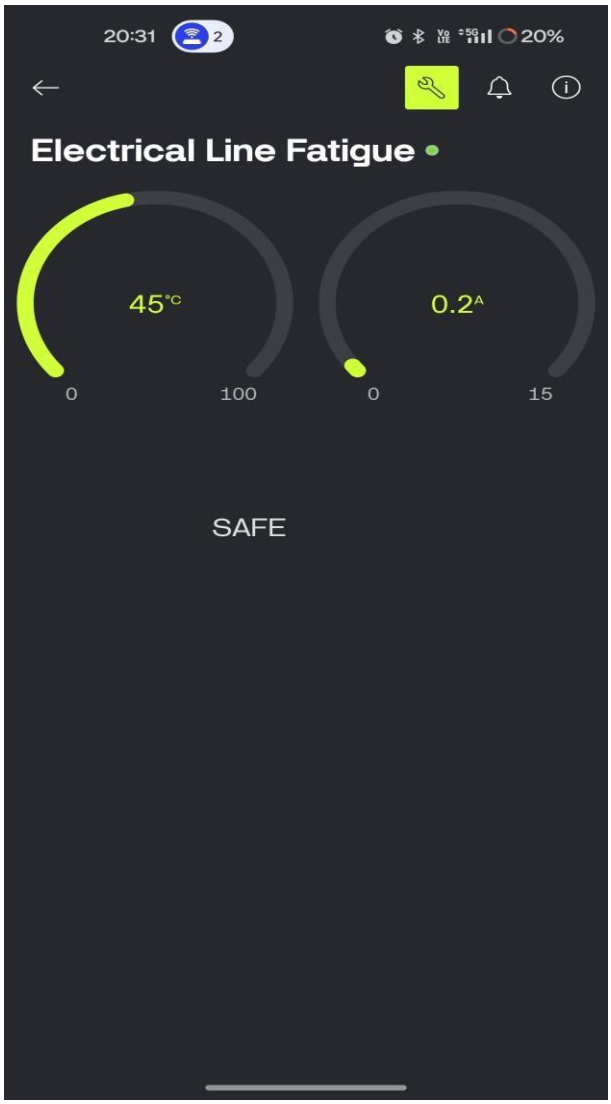
```
void loop()
{
  // Temperature
  int rawTemp = analogRead(TEMP_PIN);
  float voltage_T = (rawTemp / 1023.0) * 5.0;
  float temperature = voltage_T * 100.0;

  // Current
  int rawCurr = analogRead(CURRENT_PIN);
  float current = (rawCurr / 1023.0) * 13.5;

  // Status Logic (FIXED)
  bool isDanger  = (temperature >= TEMP_DANGER || current >=
CURR_DANGER);
  bool isFatigue = ((temperature >= TEMP_FATIGUE && temperature <
TEMP_DANGER) ||
                    (current >= CURR_FATIGUE && current < CURR_DANGER));

 String status;

  if (isDanger) {
    status = "DANGER";
  }
  else if (isFatigue) {
    status = "FATIGUE";
  }
  else {
    status = "SAFE";
  }

  // LEDs
  digitalWrite(LED_GREEN,  status == "SAFE");
  digitalWrite(LED_YELLOW, status == "FATIGUE");
  digitalWrite(LED_RED,    status == "DANGER");

  // Buzzer
  if (status == "DANGER") {
    digitalWrite(BUZZER, HIGH);
    delay(200);
    digitalWrite(BUZZER, LOW);
    delay(200);
  } else {
    digitalWrite(BUZZER, LOW);
  }
```

```
// LCD
lcd.setCursor(0, 0);
lcd.print("T:");
lcd.print(temperature, 1);
lcd.print("C ");

lcd.print("I:");
lcd.print(current, 2);
lcd.print("A ");

lcd.setCursor(0, 1);
lcd.print("Status: ");
lcd.print(status);
lcd.print("   ");

// Serial
Serial.print("Temp: ");
Serial.print(temperature);
Serial.print(" C | Current: ");
Serial.print(current);
Serial.print(" A | Status: ");
Serial.println(status);

delay(300);
```

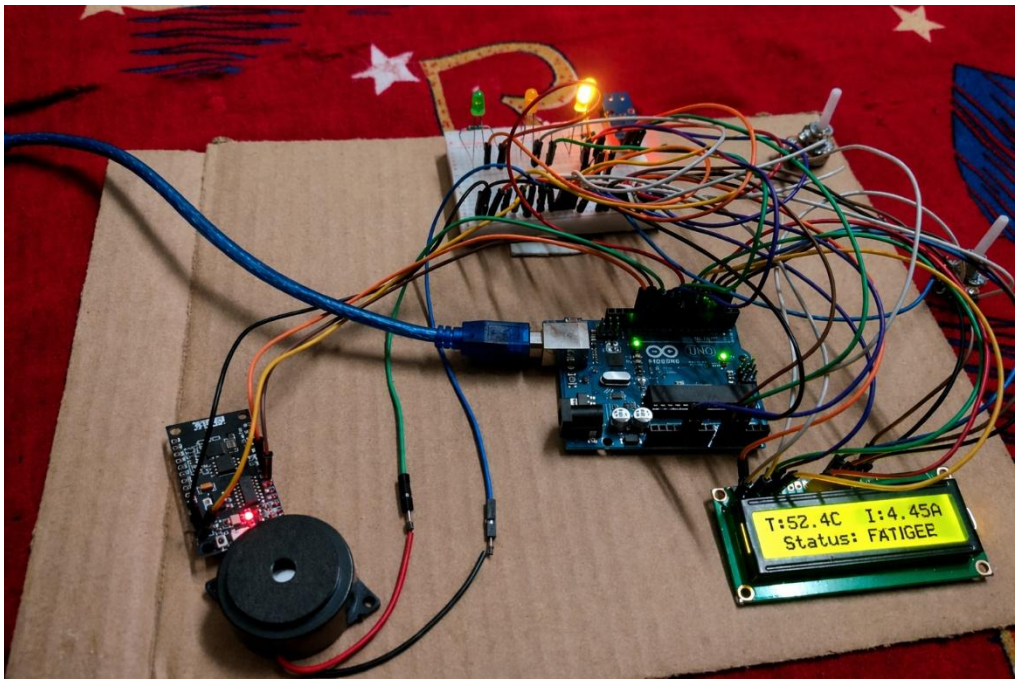
OUTPUT

Blynk App — SAFE Condition

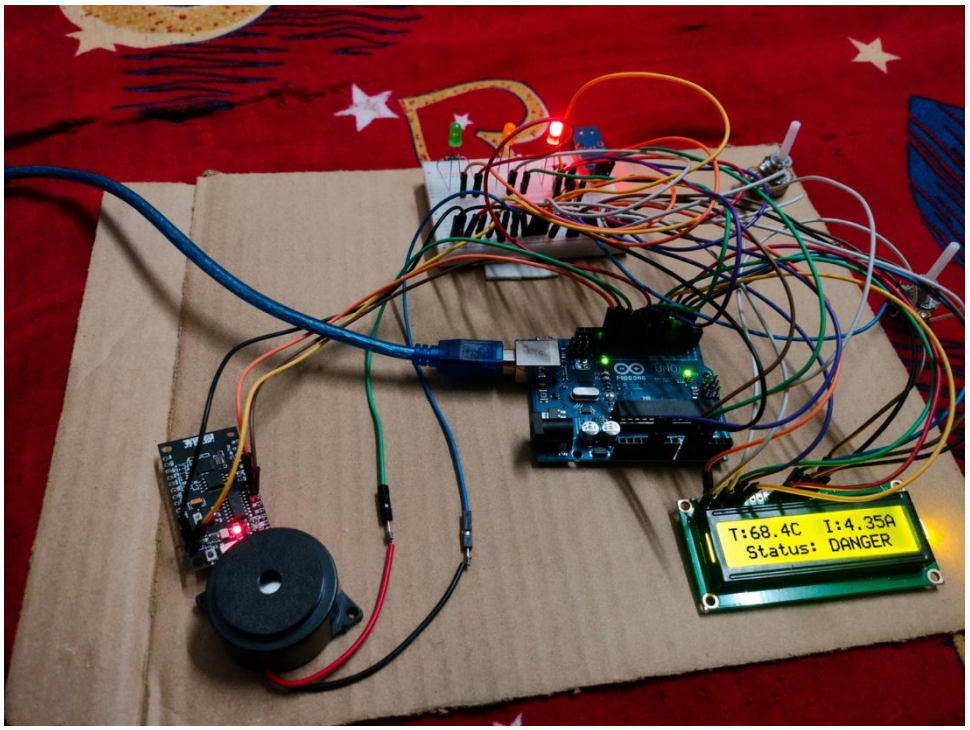


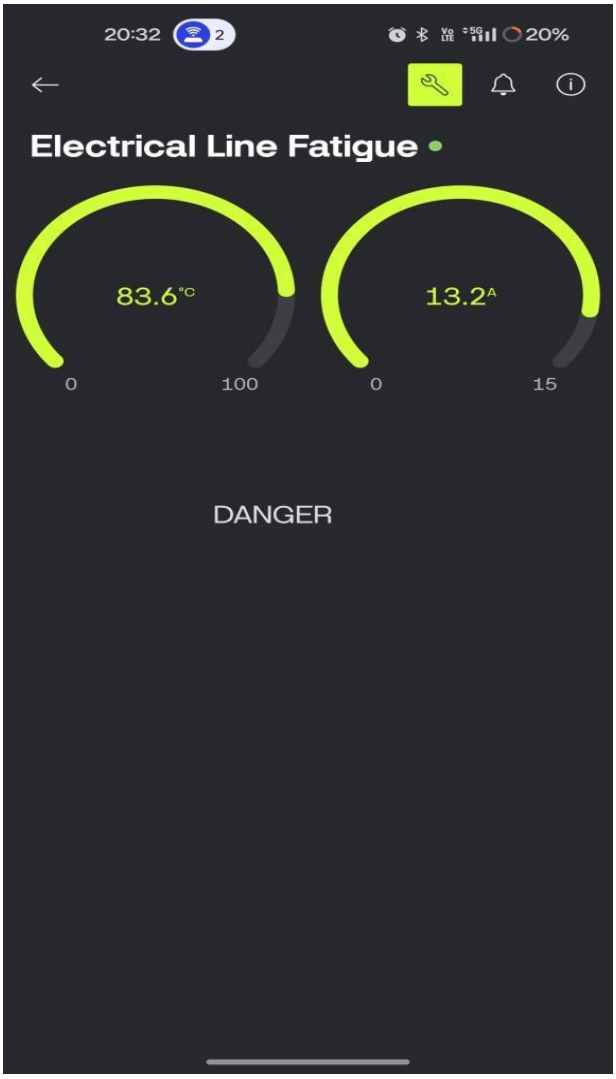


Blynk App — FATIGUE Condition



Blynk App — DANGER Condition





CONCLUSION

The Electrical Line Fatigue Monitoring System successfully monitors temperature and current of electrical lines in real time and provides instant alerts through both local indicators and the Blynk smartphone application when unsafe conditions are detected.

It effectively:

- Detects overheating and overloading conditions in electrical lines
- Classifies conditions as SAFE, FATIGUE, or DANGER using threshold logic
- Displays real-time data on a 16×2 LCD display locally
- Alerts via Green/Yellow/Red LEDs and an active buzzer
- Transmits live data to Blynk cloud via NodeMCU ESP8266 over WiFi
- Sends push notifications to smartphone on status change